

Agentic AI System Design Report

Project 6 — Agentic AI Systems *Property-Management Operator Agent*

Overview

This project delivers a multi-agent AI assistant for a small property manager — an operator who runs a few apartment buildings and handles tenant requests, maintenance scheduling, lease renewals, and rent reconciliation. The assistant ingests a request (typed text or an attached image), drafts a short plan, gates each planned action against a written policy, executes only what the policy permits, and records every step to a tamper-evident audit log.

The system is built on **LangGraph** (a state-machine framework for agent control flow) and uses **LangChain**'s message/tool abstractions. The language model is `kimi-k2.7-code:cloud` hosted on Ollama Cloud, chosen because it is natively multimodal — the same model handles text planning and image (vision) intake. The agent follows the ReAct pattern of interleaving reasoning and acting (Yao et al., 2023), materialized as distinct **planner**, **worker**, **reviewer**, **dispatch**, **summarizer**, and **human-approval** nodes with explicit control-flow edges between them, rather than a single prompt-and-call loop.

The defining design choice is the **non-bypassable policy gate**. The reviewer node is a pure rule evaluator — it loads a YAML policy and answers "is this action allowed?" with no language model in the path. Because the worker never calls a tool directly (it always routes through the reviewer first), no amount of prompt crafting can talk the agent into performing a forbidden action. An eviction or lockout request is hard-blocked; no tool runs; the rejection is logged. This is the rubric's required failure case and the system's central safety property.

A second design choice shapes the whole codebase: the governance layer (`src/governance/`) is **domain-agnostic**. It contains no property-management strings and imports nothing from the domain layer. Project 7 of the capstone — a security-operations-center (SOC) copilot — is designed to lift this governance layer verbatim and swap in a security `domain/` package, reusing the same graph shell, the same policy gate, and the same audit log. A guard test enforces the boundary so the transfer cannot break silently.

System Architecture and Design

The graph

The agent is a LangGraph `StateGraph` with a shared `AgentState` (a `TypedDict`) flowing through every node. The topology:

```
START -> ingest -> planner -> worker
worker -> reviewer (always, before any tool runs)
reviewer -> {allow: worker_dispatch, require_human: human_approval, block: summarizer}
worker_dispatch -> {more_steps: reviewer, done: summarizer}
human_approval -> worker_dispatch
summarizer -> END
```

The compiled graph is rendered to `reports/agent_graph.png` (via LangGraph's `draw_mermaid_png()`) and displayed inline in the notebook.

Node responsibilities

Node	Responsibility	LLM?
ingest	Read an attached image (OCR via <code>pytesseract</code> , with median per-word confidence; fall back to the vision LLM on low confidence or OCR failure); redact PII; place clean text in state	yes (vision fallback)
planner	Read the redacted request + recent memory; produce a 1–4 step plan (action + reason per step) as JSON	yes
worker	For the current plan step, fill the action's arguments (in stub mode it uses the plan step's pre-filled args)	yes
reviewer	Evaluate the action against the YAML policy → <code>allow</code> / <code>require_human</code> / <code>block</code> . No LLM, no tools	no
worker_dispatch	Run the approved tool from the registry; log the call; advance the step index	no
human_approval	Suspended step for actions requiring a human. Demo auto-grants; production would use LangGraph <code>interrupt</code>	no
summarizer	Produce a user-facing recap; append to memory; finalize the turn	yes (optional)

The non-bypassable gate

The edge `worker -> reviewer` is fixed and unconditional. The only way to reach `worker_dispatch` (which actually calls the tool) is through the reviewer's `allow` route. Because the reviewer contains no language model, there is no prompt-injection path from user input to tool execution for a forbidden action — the user's text reaches the planner and

worker, but the reviewer only reads the policy and the action's arguments. This is the structural realization of the principle that safety-critical checks should not depend on the same model that generates the actions (Ganguli et al., 2022).

The policy and its constraint evaluator

The policy (`domain/policy.yaml`) lists each action with `side_effect`, `allow`, optional `require_human`, optional `constraints`, optional `require_fields`, and a `block_reason` for forbidden actions. The reviewer's constraint check is a small **predicate evaluator** supporting `gt`, `ge`, `lt`, `le`, `eq`, `ne`, `in`, and `regex_match`. This is the linchpin abstraction: P6's spend cap (`cost_estimate: {le: 500}`) and P7's risk-band threshold (`risk_band_score: {ge: 75}`) are enforced by the *same* mechanism, which is what lets the governance layer stay domain-agnostic.

Read-only actions (`tenant.query`, `ledger.read`, `ledger.reconcile`) are auto-allowed and skip the human branch. Side-effect actions under their constraint pass straight through. Side-effect actions that require a human suspend at the human-approval node. Forbidden actions (`tenant.evict`, `tenant.lockout`) are `allow: false` and route directly to the summarizer with a `block` audit entry — no tool is called.

Memory

Memory is a per-user SQLite scratchpad (`governance/memory.py`): a chronological log of `user_msg`, `plan`, `decision`, `tool`, `summary`, and `block` entries for each turn. The planner injects the last few entries into its system prompt under a `<prior_turns>` block so a follow-up turn can reference an earlier one. No vector store is used — SMB property-management conversations are short, and semantic recall is unnecessary for this scope. The decision to keep memory simple rather than add a retrieval index follows the well-known guideline that one should choose the simplest architecture that meets the requirement and only add complexity when a concrete need forces it (Yao et al., 2023; Chase, 2024). P7's RAG retrieval over SOPs will be a separate component, not this memory.

The audit log

Every decision, tool call, block, and human approval is appended to a JSONL file by `governance/audit.py`. Each line carries `prev_hash` and `this_hash`, where `this_hash = sha256(prev_hash + canonical_json(line_without_this_hash))`. The first line chains from a genesis hash. `verify_chain()` recomputes every hash from genesis and raises `ChainBrokenError` at the first mismatch, so editing any past entry breaks the chain. This hash-chained, append-only structure is the same integrity technique used by tamper-evident logs and blockchain-style ledgers (Nakamoto, 2008); it gives the system a defensible "this is exactly what the agent did, and nobody altered it" property that a plain log file cannot claim. Even

read-only tool calls are logged — the trail records what was inspected, not just what was changed, which matters for post-incident review.

Multi-modal intake

The intake node (`domain/intake_node.py`) reads an attached image with `pytesseract`, computes the **median** per-word confidence (the median rather than the mean because garbage words skew the mean), and falls back to the `kimi-k2.7-code:cloud` vision model when confidence is below the policy threshold (65) or when OCR fails (e.g., the Tesseract binary is missing). Either way, PII is redacted by `governance/pii.py` (regex patterns from the policy) *before* the text reaches the planner, memory, or the audit log. The separation of redaction into a domain-agnostic module driven by policy patterns means P7 can add badge/employee-ID redaction by editing the YAML, not the code.

Design tradeoffs

- **Stub mode vs. LLM mode.** Scenarios 1–4 inject a scripted plan and run with `llm=None`, so the demo is deterministic and runs anywhere (CI, a reviewer's laptop without credentials). Scenario 5 runs the real model on a sample image. This trades realism for reproducibility in the bulk of the demo — the deterministic paths are the ones the tests assert.
- **Auto-granted human approval.** The human-approval node logs the decision but auto-grants in demo mode, so the graph runs end-to-end without a human present. A production deployment would call LangGraph's `interrupt` and block until a human responds (Chase, 2024). This is the chief honesty gap between the demo and a deployable system.
- **`tool_specs=[]` in the demo.** The worker has the action name from the plan but no bound tool schema, so its per-step arg-filling can drift to the read-only tool even when the planner named a different action. The reviewer still gates every step correctly; binding real tool schemas is the documented next step.

Decision Logic and Behavior

The agent's decision logic is split across three layers, in increasing order of authority:

1. **The planner (LLM)** decides *what* to attempt — a sequence of actions. Its system prompt enumerates the allowed actions and explicitly forbids planning evictions or lockouts. But the planner is not trusted: a plan is a proposal, not a commitment.
2. **The worker (LLM)** decides *how* to fill an action's arguments. In stub mode the plan step's pre-filled args are used; in LLM mode the model produces a JSON object the

parser tolerantly interprets.

3. **The reviewer (no LLM)** decides *whether* the action may run. This is the only layer with authority over tool execution, and it is unreachable by prompt injection.

Observed behavior across the five scenarios

1. **Read-only happy path** — "What is the lease status of unit 4B?" → planner → reviewer auto-allows → `tenant.query` runs → summarizer. Audit trail: a `decision` (planner), a `decision` (reviewer allow), a `call` (`tenant.query`), a `decision` (turn complete). Chain verifies.
2. **Side-effect with spend cap** — "Schedule maintenance for unit 4B, \$300" → the reviewer's `le: 500` predicate passes → `maintenance.schedule` runs and writes a row. Audit trail includes the `call` with the result summary.
3. **Human approval** — "Renew the lease for unit 4B" → policy marks `lease.renew` `require_human: true` → graph routes to the human-approval node → (auto-granted in demo) → `lease.renew` runs. Audit trail shows the `human_approval` entry followed by the `call`.
4. **Failure case — eviction/lockout** — "Lock out tenant T-0042" → the reviewer returns `allow=False`, `violations=['action_not_allowed']` → graph routes straight to the summarizer → **no tool is called**. The audit trail contains a `block` entry with the policy's `block_reason`. The test `test_eviction_blocked.py` asserts that no `tenant.lockout` `call` appears in the log — the proof the gate held.
5. **Multi-modal LLM run** — `kimi-k2.7-code:cloud` reads `data/sample_request.png` (a typed late-rent reminder letter) via the vision fallback (Tesseract is not installed on the build machine), the planner drafts a three-step plan (`ledger.read`, `tenant.query`, `ledger.reconcile`) from the transcribed text, and the reviewer gates and dispatches each step.

The eviction scenario is the keystone: it shows the agent refusing an action that a pure LLM assistant might rationalize ("the tenant is behind on rent, so locking them out is reasonable"). The reviewer does not reason about the tenant's situation; it only checks the action against the policy. That is the intended behavior for a system that can take real actions on people.

Safety, Reliability, and Transparency

The system includes several safeguards, each with a concrete purpose:

- **Non-bypassable policy gate.** The reviewer is a pure function of (action, args, policy). It contains no LLM, so it cannot be manipulated by user input. Forbidden actions never

reach a tool. This is the primary safety mechanism and is regression-tested by `test_eviction_blocked.py`.

- **Fail-closed on missing constraints.** A side-effect action whose required fields are missing, or whose constraints are unsatisfied, is blocked and routed to a human rather than silently dropped or auto-approved with default values. The policy never defaults a side-effect cost to zero.
- **Hash-chained audit log.** Every step is recorded; tampering is detectable. `verify_chain()` is exercised in `test_audit_chain.py` (clean chain passes; a tampered line raises `ChainBrokenError`). This gives post-incident review a trustworthy record — the operator can show *why* a critical action was or was not taken.
- **PII redaction at the boundary.** Redaction runs in the intake node before any other component sees the text. The planner, memory, and audit log only ever see redacted text. This is a privacy safeguard and a scope-minimization principle: the system operates on the minimum information needed (US NIST, 2020).
- **OCR→vision fallback.** When OCR is unavailable or low-confidence, the intake node routes the image to the vision LLM rather than silently producing empty text. The route taken is itself logged (`ocr_route:unavailable`, `ocr_route:vision_fallback_on_error`), so the provenance of every piece of intake text is inspectable.
- **Domain-agnosticism guard.** `test_governance_no_domain_imports.py` fails if any governance module imports `domain`. *or contains property-management string literals.* *This is a structural safeguard for the P7 transfer: it makes "the governance layer is reusable" a tested* claim rather than an assertion.*

Transparency is served by the audit log (every decision is recorded with its reason), the policy being a human-readable YAML file (the rules are not buried in code), and the graph diagram (`reports/agent_graph.png`) making the control flow inspectable.

Observed Behavior and Limitations

What works

- All five scenarios run end-to-end; 21 tests pass. The eviction failure case is demonstrated and asserted: the blocked action never reaches a tool.
- The hash chain verifies after every run and detects tampering in the test suite.
- The LLM image scenario shows the full multimodal pipeline: image → vision transcription → PII redaction → planning → gated execution, with the audit trail recording the OCR-route decision and every step.
- The governance layer is provably domain-agnostic (the guard test passes), supporting the P7-transfer claim.

Limitations

1. **Human approval is auto-granted in demo mode.** A real deployment would block via `LangGraph interrupt` and wait for a human. The audit entry is logged, but no human actually reviewed it. This is the largest gap between the demo and a deployable system.
2. **Worker tool selection in LLM mode is loose.** Because `tool_specs=[]` in the demo, the worker's per-step arg-filling can drift to the read-only tool even when the planner named a different action. The reviewer still gates every step correctly, so this is a *quality* limitation (the wrong action may run) not a *safety* one (a forbidden action still cannot run). Binding real tool schemas to the worker would fix it.
3. **OCR depends on the Tesseract binary.** Without it, intake falls back to the vision LLM, which requires credentials and costs API calls. The system degrades gracefully but the OCR path is not exercised in this build.
4. **The policy is static.** It is loaded from YAML at startup; there is no runtime policy update or per-tenant override. Adequate for a demo, not for a multi-tenant production system.
5. **No persistence of incidents across runs** beyond the audit log and scratchpad. A production system would likely maintain a durable case store; here the SQLite tools re-seed on each fresh `data/` directory.

Failure cases observed

- The eviction/lockout block (Scenario 4) is the deliberate failure case.
- An early integration bug — the planner's LLM returned JSON wrapped in a

````json```` fence, which the parser rejected, producing an empty plan — was fixed by stripping markdown fences in `parse_plan`. This is documented as a reminder that LLM output parsing must be tolerant of the formatting models commonly add.

---

## Ethical and Responsible Use Considerations

The most salient ethical concern for this system is **accountability** for an agent that can take actions on people — specifically, tenants. A property manager's assistant that can schedule maintenance, renew leases, and (in a misdesigned system) initiate lockouts has real-world consequences for people's homes. The central ethical design decision is therefore not "make the agent smarter" but "make the agent unable to do the things it must never do without a human."

The non-bypassable policy gate operationalizes this: the eviction/lockout refusal is not a polite suggestion from the model — it is a structural impossibility. The reviewer cannot be convinced, because it does not reason. This reflects the principle that high-stakes decisions in automated

systems should preserve meaningful human control, and that automation should not be used to launder responsibility for decisions that require human judgment (Floridi & Cowls, 2019; US NIST, 2020).

A second concern is **privacy and scope minimization**. The system handles tenant contact details and payment status. PII redaction at the intake boundary ensures the model and the logs see only what they need; the audit log stores redacted text and truncated result summaries, never raw payloads. Retention limits are declared in the policy (`retention.audit_log_days`, `memory_days`). This is a specific, grounded implementation of data minimization, not a generic privacy disclaimer.

A third concern is **transparency and contestability**. Because every decision is written to a tamper-evident log with its reason, a tenant or manager who disputes an outcome can ask "why was this done?" and receive a verifiable answer rather than an inscrutable model output. The audit trail makes the agent accountable in the literal sense — its actions can be accounted for.

A fourth concern, relevant to the P7 transfer, is **"bias in automated decisions"**. The P6 policy's spend cap and required fields are explicitly chosen and inspectable; they are not learned from data, so they cannot silently encode biased thresholds. The P7 risk-scorer, by contrast, *will* be learned — and the same governance gate that blocks eviction here will be the place where P7's bias audits attach (e.g., checking that escalation thresholds do not disadvantage particular sites or user roles). The discipline of "rules first, ML second" inherited from earlier capstone work is, in part, an ethical choice: interpretable, auditable rules before opaque models.

Finally, the **limitations above are themselves an ethical matter**: auto-granting human approval in the demo means the "human-in-the-loop" claim is not yet operational. Reporting this honestly — rather than implying the human gate is live — is a responsible-use obligation. A deployment that silently auto-approved critical actions while claiming human review would be deceptive.

---

## Future Improvements

---

1. **Real human-in-the-loop via LangGraph `interrupt`**. Replace the auto-grant with a genuine suspend/resume that blocks until a human responds, with a timeout and an escalation path. This closes the largest demo-vs-production gap.
2. **Bind real tool schemas to the worker**. Provide `StructuredTool` specs so the worker fills args from a schema rather than free-form JSON, eliminating the per-step tool-selection drift.
3. **Runtime policy reload and per-tenant overrides**. Allow the policy to be updated without restarting the agent, and scope certain rules to specific buildings or lease types.



4. **Persistent case store.** A durable record of incidents/turns beyond the audit log, so a returning user sees continuity across sessions.
  5. **Structured evaluation of the LLM paths.** A held-out set of requests with expected plan/action outcomes, reported as precision/recall per action — the deterministic paths are tested, but the LLM planning and arg-filling are only qualitatively evaluated today.
  6. **P7 transfer validation.** An automated check that builds the SOC domain/ package and confirms the governance graph compiles and routes correctly with it — turning the "P7 can reuse this" claim into a passing test.
  7. **Redaction enhancement.** Move beyond regex to a named-entity-based redactor for harder cases (free-text notes with names and addresses), while keeping the policy-driven interface so the rules remain inspectable.
- 

## References

---

Chase, H. (2024). LangGraph [Computer software]. GitHub. <https://github.com/langchain-ai/langgraph>

Floridi, L., & Cowls, J. (2019). A unified framework of five principles for AI in society. *Harvard Data Science Review*, 1(1). <https://doi.org/10.1162/99608f92.8cd550d1>

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system*. <https://bitcoin.org/bitcoin.pdf> (hash-chained, tamper-evident logging technique).

Ollama. (2026). *kimi-k2.7-code model card*. Retrieved from <https://ollama.com/library/kimi-k2.7-code>

Ganguli, D., Lovitt, L., & Kernion, J. (2022). *Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned*. <https://arxiv.org/abs/2209.07858>

US National Institute of Standards and Technology. (2020). \*NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management.\* Version 1.0. <https://www.nist.gov/privacy-framework>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR 2023). <https://arxiv.org/abs/2210.03629>